



Fabric IQ e le ontologie

Dal dato alla lingua del business: cosa sono le ontologie, perché servono all'AI, come si costruiscono in Microsoft Fabric e chi le consuma





Marco Pozzan

- Fondatore e CTO Regolo Farm azienda di Business Intelligence e Artificial Intelligence
- Fondatore e CTO Cargo BI software di analisi con machine learning in campo assicurativo
- Membro del comitato etico di Enia (Ente Nazionale Intelligenza Artificiale)
- Docente all'Università di Pordenone per i corsi IFTS di analisi dati e algoritmi predittivi
- MCP,MCSA,MCSE dal 2017 MCT e dal 2014 MVP per SQL Server 2016 MVP Power BI e dal 2022 MVP Fabric
- Marco.pozzan@regolofarm.com
- [@marcopozzan.it](https://www.linkedin.com/company/marcopozzan) (linked-in)
- [@regolo farm](https://www.linkedin.com/company/regolo-farm) (linked-in)
- www.marcopozzan.it

Agenda

- Fabric IQ
- Le ontologie
- Come si costruiscono
- NL2Ontology
- Chi le consuma
- Demo
- Arricchimento semantico

Fabric IQ fa parte di Microsoft IQ

- Microsoft IQ è il livello di intelligenza aziendale dello stack Microsoft: quattro capacità che insieme danno agli agenti una visione completa dell'organizzazione

Fabric IQ

Contesto su entità di business e dati
(Microsoft Fabric)

Work IQ

Contesto su come lavorano le persone
(Microsoft 365)

Foundry IQ

Contesto su policy e documenti autorevoli
(Azure AI Foundry)

Web IQ

Contesto proveniente dal web
(Bing search)

- **Il punto di Fabric IQ:** È radicato in tutti i dati analitici, real-time e operativi dell'azienda. Il dato da solo non basta e Fabric IQ eleva quel dato al linguaggio del business.
- **La conseguenza pratica:** Persone e agenti interpretano correttamente i dati e ragionano in termini di concetti e obiettivi aziendali, non di righe e colonne.

I tre livelli di Fabric IQ

I tre livelli sono veicolati da 2 item core del workload IQ: **ontology** (preview) e **semantic model**.

- **Ontology (preview):** Definisce **entità, proprietà, relazioni, regole e azioni**. Gli agenti capiscono quali azioni sono disponibili e come invocarle.
- **Power BI semantic model (business Intelligence):** Livello analitico curato con misure, gerarchie e dimensioni. Le ontologie possono essere generate direttamente da qui.
- **OneLake(Unified data) :** Fondamenta: dati aziendali unificati tra cloud e on-premises in un'unica fonte governata, via shortcut e mirroring.

Gli item del Fabric IQ

Item Fabric	A cosa serve	Quando lo usi
Ontology (preview)	Vocabolario condiviso: entity type, relazioni, proprietà, regole	Consistenza cross-dominio, governance, grounding degli agenti
Power BI semantic model	Modello analitico curato: misure, gerarchie, relazioni	KPI affidabili e visual interattivi per il business
Graph	Dati connessi: nodi, archi, traversal	Domande su dipendenze, catene di impatto, percorsi minimi
Data agent	Analista virtuale che risponde in linguaggio naturale	Q&A di dominio, pubblicabile su M365, Foundry, Copilot Studio
Operations agent	Monitoraggio real-time con azioni raccomandate	Rilevare anomalie e innescare risposte governate su dati live

- **Graph è integrato nell'ontologia:** Il grafo delle istanze che vedi dentro l'item Ontology è fornito da Graph in Microsoft Fabric: l'ontologia dichiara cosa si collega e perché, Graph memorizza e attraversa i collegamenti.

Che cos'è un'ontologia

Il termine viene dal greco **ontos** (l'essere, ciò che esiste) e **logos** (discorso, studio): letteralmente lo studio di ciò che esiste.

“quali entità esistono nel mio dominio, come le chiamo e come si relazionano” in modo che anche una macchina lo capisca.

Un vocabolario condiviso e comprensibile dalle macchine del tuo business:

- le **cose** che esistono nel tuo ambiente, (**entity type**): Cliente, Ordine, Impianto, Sensore
- **i fatti** che le descrivono (**property**): nome, email, temperatura, data
- **modi in cui si collegano** (**relationship**): Cliente effettua Ordine
- **i vincoli** e le **regole** che mantengono coerenti queste rappresentazioni.

Ontologia e modello dati non sono la stessa cosa

Non è un'alternativa: l'ontologia si appoggia ai dati esistenti tramite i data binding, senza copiarli e senza sostituire il modello sottostante.

Modello relazionale / star schema

- Ottimizzato per l'esecuzione efficiente delle query
- Il significato vive nella join logic e nella documentazione
- La stessa entità è definita in modo diverso in ogni sorgente
- Ogni domanda nuova richiede la traduzione di un esperto
- Le relazioni sono foreign key: nomi tecnici, senza semantica

Ontologia

- Ottimizzata per il significato e per il ragionamento
- Il significato è esplicito, dichiarato e interrogabile
- Il concetto è definito una volta sola e riutilizzato ovunque
- La domanda si pone in termini di business, non di tabelle
- Le relazioni sono tipizzate e direzionali: monitora, fornisce, si trova in

Come i dati entrano nell'ontologia

Data binding

Collega le definizioni ai dati concreti in OneLake: tabelle lakehouse, stream eventhouse, semantic model Power BI. Descrive tipi, chiavi di identità, mappatura colonne → proprietà. Integra senza copiare i dati sorgente.

Ontology graph

Grafo di istanze interrogabile, costruito dai binding e dalle relazioni, fornito da Graph in Fabric. I nodi sono istanze, gli archi sono legami con metadati. Abilita esplorazione visiva, algoritmi su grafo e inferenze.

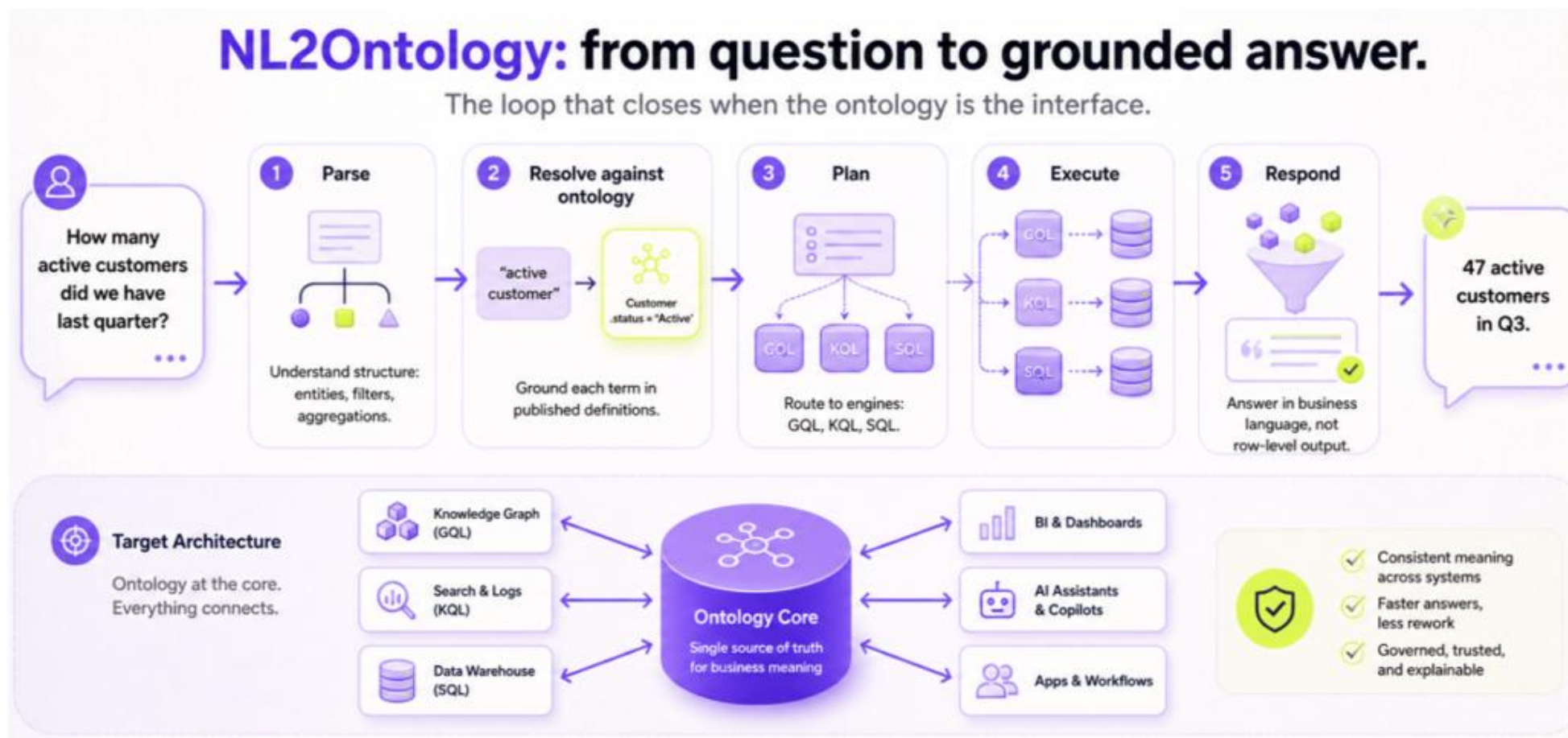
Querying / NL2Ontology

Domande a livello di business sui termini dell'ontologia. Il layer instrada la query al motore più efficiente (GQL per Graph, KQL per Eventhouse). NL2Ontology traduce le domande in linguaggio naturale in query strutturate.

Quando un utente chiede "Quanti clienti attivi abbiamo avuto lo scorso trimestre?", il livello NL2Ontology non deve indovinare cosa significhi "cliente attivo". Legge la definizione dall'ontologia. **Non deve indovinare come collegare i clienti agli ordini.** La relazione è dichiarata.

Il risultato è una query strutturata — in GQL, KQL, SQL o una combinazione di questi — che si basa sulle definizioni dell'ontologia.

NL2Ontology, quando arriva una domanda:



Vantaggi ontologia per un agente

- **Evita ambiguità:** Cosa significa "cliente attivo"? Tre reparti, tre definizioni, tre risposte diverse alla stessa domanda.
- **Evita traduzione manuale:** Senza comprensione semantica ogni domanda richiede un esperto di dominio che la traduca in SQL o DAX.
- **Nessuna spiegabilità(Governance):** Una risposta generata su colonne anonime non è verificabile: non si sa da dove viene né cosa ha assunto.
- **Non adatta a decisioni:** Per questo l'AI resta confinata ai casi a basso rischio e non entra nei processi decisionali critici.

Due strade per creare un'ontologia

Nella pratica si usano entrambe: si genera da semantic model per partire in fretta, poi si estende e si rifinisce a mano.

A · Generazione automatica

da un semantic model Power BI

Quando sceglierla

Hai già un semantic model ben strutturato che rappresenta il tuo dominio e vuoi partire da logiche e definizioni già in produzione.

Vantaggio

Bootstrap immediato e coerenza terminologica tra report, agenti e applicazioni.

B · Costruzione manuale

direttamente da OneLake

Quando sceglierla

Non hai un semantic model, oppure vuoi controllo completo sul disegno dell'ontologia fin dall'inizio.

Vantaggio

Modelli il dominio come lo intende il business, senza ereditare i compromessi tecnici del modello analitico.

Generazione automatica

Genera automaticamente

- Un nuovo item Ontology nel workspace
- Gli entity type, uno per ogni tabella del semantic model
- Le proprietà statiche a partire dalle colonne
- I data binding che collegano le righe alle proprietà
- I relationship type che seguono le relazioni del modello

Resta da fare a mano

- Legare i dati time series: quelle proprietà non si creano da sole
- Rivedere le entity type key e aggiungere quelle mancanti (tipico sulle tabelle dei fatti)
- Legare i relationship type ai dati: sono definiti ma non configurati
- Rinominare le entità: arrivano col nome della tabella (dimstore, factsales)

Il percorso manuale in quattro fasi

L'ordine non è negoziabile: senza entity type non ci sono binding, senza binding statico non c'è time series, senza chiavi non ci sono relazioni.



Entity type: definire i concetti

Regole sui nomi: Da 1 a 26 caratteri, solo alfanumerici, trattini e underscore, primo e ultimo carattere alfanumerico. Vale per entità e proprietà.

Proprietà non tipizzate: Con "Define at binding" definisci il concetto senza decidere subito il tipo: utile per separare il modello concettuale da quello fisico. Restano invisibili a query e strumenti downstream finché il binding non assegna loro un tipo.

Data binding: collegare i dati reali

Il binding integra i dati nel layer semantico senza copiarli, arricchisce le entità con informazioni aggiornate da fonti batch e real-time, e fornisce la spina dorsale semantica per gli agenti.

Dati statici (per primi)

Definiscono le istanze uniche dell'entità. Nel wizard: Entity type key, Binding selection, Entity type key mapping e Properties. Le colonne sorgente popolano automaticamente la mappatura.

Serie temporali (dopo)

Richiedono che il binding statico sia già completo: serve almeno una proprietà statica da usare come chiave per contestualizzare le misurazioni, e deve corrispondere esattamente a una colonna dei dati time series.

Data binding: limiti

Limiti da conoscere prima di progettare:

- 1) Un solo binding statico per entity type: non si combinano più sorgenti statiche.
- 2) Più binding time series sì: possono arrivare sia da eventhouse sia da lakehouse.
- 3) Solo tabelle lakehouse gestite (managed), non tabelle esterne.
- 4) Niente lakehouse con OneLake security abilitata

Relationship type: rendere esplicito il contesto

Un esempio: Relazione "drives" da Driver a Truck, basata sulla tabella Truck data: le colonne DriverId e TruckId generano un'istanza di relazione per ogni combinazione presente nei dati.

Perché conta: Le relazioni rendono il contesto esplicito e riutilizzabile: abilitano traversal, analisi delle dipendenze e inferenza basata su regole, senza scrivere join custom ogni volta

Chi usa le ontologie

L'ontologia dà agli agenti una comprensione condivisa e governata del business: entity type, relazioni, definizioni, regole e mappature verso le sorgenti.

- **Fabric data agent:** Q&A conversazionale dentro Fabric (**Analisti e utenti business**)
- **Fabric operations agent:** Monitoraggio continuo e azioni raccomandate (**Team operations**)
- **Copilot Studio:** Agenti conversazionali low-code (**Business maker**)
- **Foundry IQ:** Agenti custom con tool calling (**Sviluppatori**)
- **Client MCP:** Qualsiasi client AI compatibile MCP (**Sviluppatori**)

Quale agente scegliere

Il data agent risponde a domande: è interattivo, lo interroga una persona.
L'operations agent osserva: gira in continuo sui dati live, rileva condizioni e notifica o agisce (per esempio su Teams).

Agente	Esperienza principale	Ideale per
Fabric data agent	Q&A conversazionale dentro Fabric	Analisi interattiva su dati governati con contesto ontologico
Fabric operations agent	Monitoraggio continuo con azioni raccomandate	Alerting e azioni automatiche rispetto <u>a</u> obiettivi di business
Foundry IQ agent	Agente developer con tool calling	Agenti avanzati integrati con sistemi enterprise fuori da Fabric
Copilot Studio agent	Agente conversazionale low-code	Agenti business-friendly e automazione di workflow senza codice
Custom via MCP server	Qualsiasi client AI compatibile MCP	Collegare sistemi AI esterni o costruiti in casa

Copilot e ontologie: cosa vale davvero

"Copilot" indica esperienze diverse e solo alcune arrivano all'ontologia. Vale la pena distinguerle prima di prometterlo a un cliente 😊.

- **Copilot Studio (Sì, DIRETTAMENTE):** L'agente collega l'ontologia tramite il tool Fabric IQ Ontology MCP e ne ricava struttura dati e contesto di business. Servono i tool MCP abilitati nell'ambiente e un item Ontology accessibile dal catalogo OneLake.
- **Microsoft 365 copilot (Sì, TRAMITE DATA AGENT):** Crei un data agent che ha l'ontologia come sorgente e lo pubblichi nell'Agent Store. In Copilot lo usi come qualsiasi agente, in chat diretta o richiamandolo con @. Serve licenza M365 Copilot e stesso tenant.
- **Copilot dentro Fabric (NO):** I Copilot di Fabric assistono su attività interne al prodotto generare codice in un notebook, scrivere query nel warehouse. Non conoscono la tua ontologia e non la interrogano.

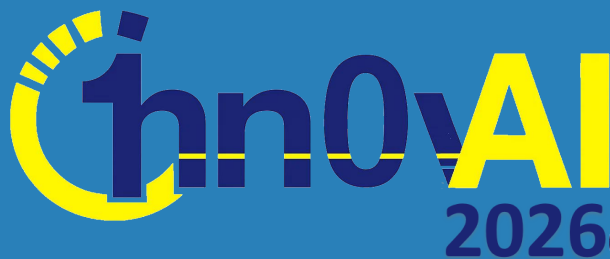
L'ontologia come server MCP

Il Model Context Protocol permette ai sistemi AI di scoprire e utilizzare strumenti esterni in modo strutturato. L'ontologia può funzionare da server MCP, esponendo un'API attraverso cui qualsiasi client compatibile può interrogarla.

```
https://api.fabric.microsoft.com/v1/mcp/dataPlane/  
workspaces/{workspace-ID}/items/{ontology-item-ID}/  
ontologyEndpoint
```

Chi può collegarsi: Copilot Studio tramite i tool Fabric IQ Ontology MCP, agenti Foundry, VS Code in Agent Mode e qualsiasi client che parli MCP su HTTP con un bearer token Fabric valido.

Cosa non cambia: Il protocollo non indebolisce il perimetro di sicurezza: permessi, RLS e policy restano quelli di Fabric.



Demo

Adesso costruiamola





Demo

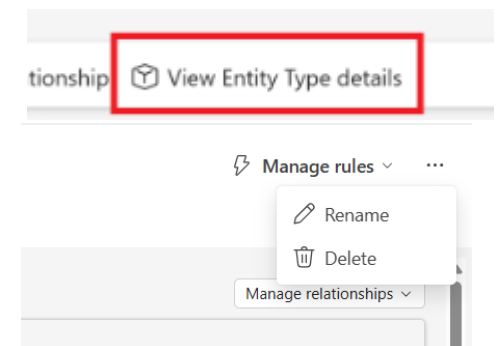
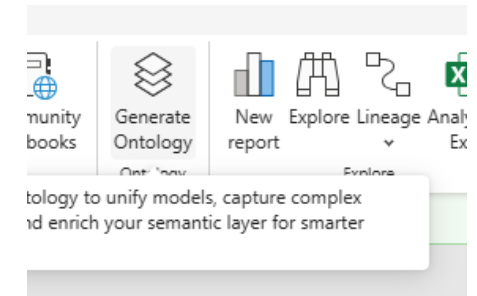
Parte 1 di 4 — Creare un'ontologia

Due percorsi per creare l'ontologia

- **Da modello semantico:** si genera automaticamente l'ontologia da un modello semantico Power BI esistente (RetailSalesModel) — ideale se il dominio è già ben rappresentato
- **Da OneLake:** si costruisce manualmente collegando le proprietà direttamente alle tabelle del lakehouse — ideale per il pieno controllo sulla progettazione
- **Scenario dell'esercizio:** Lakeshore Retail, un caso di vendita al dettaglio con negozi, prodotti e vendite
- **Nome dell'ontologia:** RetailSalesOntology — solo lettere, numeri e underscore, niente spazi o trattini

Generare l'ontologia dal modello semantico

- Aprire il modello semantico RetailSalesModel in Fabric e selezionare Genera ontologia dalla barra multifunzione (disponibile anche dalla pagina Panoramica, senza aprirlo)
- Selezionare l'area di lavoro, assegnare il nome RetailSalesOntology, Crea
- **Import automatico:** vengono creati tre tipi di entità dalle tabelle **dimproducts**, **dimstore**, **factsales**
- **Rinominare le entità (... → Rinomina):**
 - dimproducts → Products (plurale, evita conflitti con la parola riservata GQL PRODUCT),
 - dimstore → Store,
 - factsales → SaleEvent



Verificare proprietà e definire la chiave mancante

- Per ogni entità, aprire View Entity Type e controllare proprietà e origine dati
- **Products** (chiave ProductId): Brand, Category, ProductName, Subcategory
- **Store** (chiave StoreId): City, Latitude, Longitude, Region, StoreName

tionship [View Entity Type details](#)

The screenshot shows the 'Configure' tab for an entity type. It has a search bar 'Search by property name' and a dropdown menu 'Entity type key' set to 'ProductId'. Below is a table with columns 'Property name', 'Type', and 'Data source'.

Property name ↑	Type	Data source
Brand	String	dimproducts
Category	String	dimproducts
ProductId	String	dimproducts
ProductName	String	dimproducts
Subcategory	String	dimproducts

- **SaleEvent**: va definita a mano con Definisci chiave del tipo di entità → SaleId

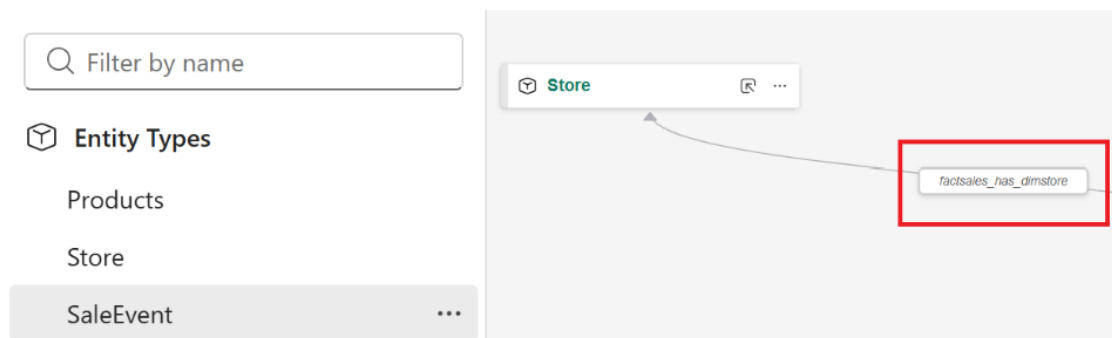
The screenshot shows the 'Entity type key' dropdown menu with the option 'Define entity type key' selected.

Property name ↑	Property source
ProductId	Local

The screenshot shows the 'SaleEvent' entity type configuration. It has a dropdown menu 'Entity type key' with the option 'Define entity type key' selected.

Configurare i tipi di relazione

Le relazioni importate dal modello semantico sono definite ma non completamente configurate



The configuration form for the relationship 'factsales_has_dimstore' is shown. It includes the following fields:

- Relationship name: from
- Mapping table: factsales
- Matched SaleEvent: SaleId (with a dropdown menu showing '123 SaleId')
- Matched Store: StoreId (with a dropdown menu showing 'ABC StoreId')

- **Attenzione:** le colonne "Matched" devono sempre corrispondere esattamente alle proprietà chiave delle due entità coinvolte

Nome	Nuovo nome	Tabella mapping	Evento di vendita	Entità target
<i>factsales_has_dimstore</i>	from	<i>factsales</i>	<i>SaleId</i>	<i>StoreId</i>
<i>factsales_has_dimproducts</i>	sold	<i>factsales</i>	<i>SaleId</i>	<i>ProductId</i>



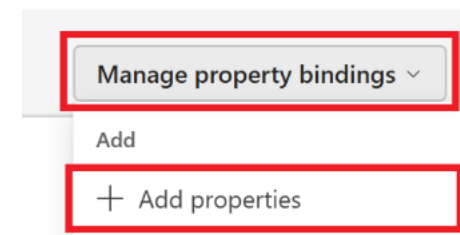
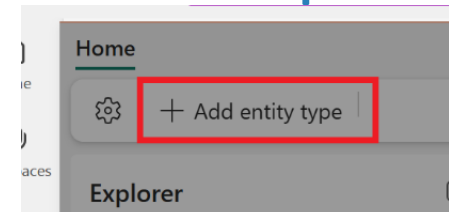
Demo

Parte 2 di 4 — Aggiunte manuali



Creare l'entità Freezer e le sue proprietà

- Aggiungi tipo di entità → nome **Freezer**
- Da View Entoty Type Details → Manage Property bindings

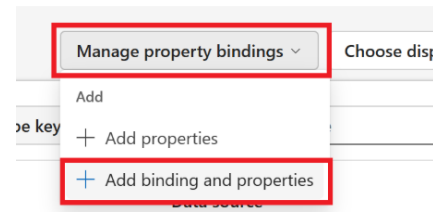


Nome	Tipo
FreezerId	string
Model	string
minSafeTempC	double
StoreId	string

- Le proprietà compaiono in configurazione ma restano "non associate" finché non si esegue il binding

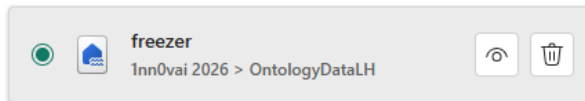
Due data binding: statico e time series

- **Binding statico:**



- **Binding time series:** Eventhouse TelemetryDataEH, tabella FreezerTelemetry

Binding selection



Add data binding

- Lakehouse table
- Eventhouse table or materialized view

Entity type key mapping

Choose source column data to map it to property name(s) used in the entity type key

Timeseries data

One or more columns have been identified as date/time

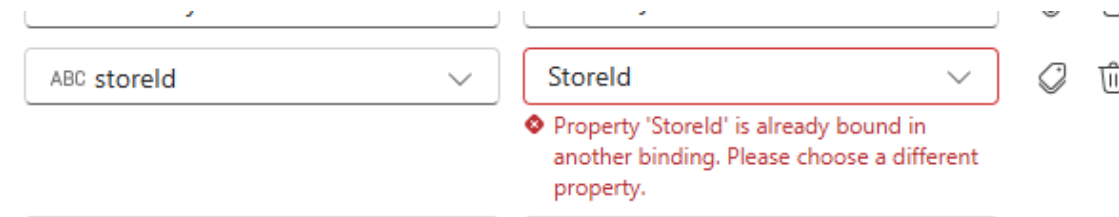
Timestamp column

Select timestamp column

timestamp

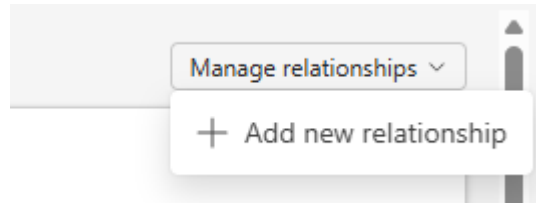
Eventhou:

- **Attenzione:** StoreId compare 2 volte (statico + time series) va eliminata la proprietà duplicata generata dal binding time series



Collegare Store e Freezer

- Aggiungi nuova relazione: nome **operates**, origine Store, destinazione Freezer



Relationship type name *

operates

Origin entity type *

Store

Target entity type *

Freezer

Next: You'll complete the data definition for this new relationship by selecting the source table that explicitly defines the relationship between your entities and choosing the key columns for each entity type.

- **Tabella di mapping:** freezer

Relationship name

operates

Mapping table ⓘ

freezer

Store

Matched Store: StoreId ⓘ *

ABC StoreId

Freezer

Matched Freezer: FreezerId ⓘ *

ABC FreezerId



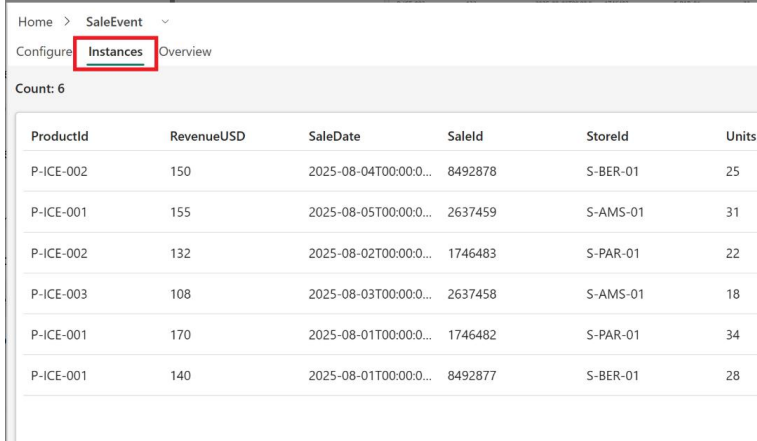
Demo

Parte 3 di 4 —

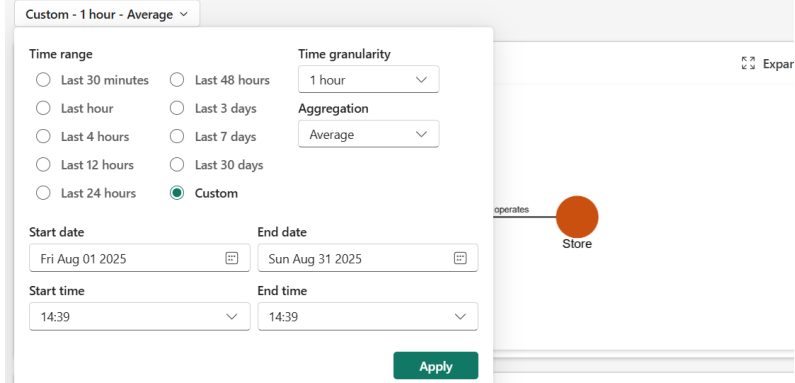
Visualizzare l'ontologia

Istanze di entità e dati time series

- Sul tipo di entità SaleEvent → scheda Istanze: 6 istanze popolate dai dati di factsales (ricavi, unità vendute)
- Passare al tipo di entità Freezer → scheda Panoramica: i grafici sono vuoti con l'intervallo predefinito (ultimi 30 giorni)
Impostare un intervallo personalizzato (es. 1-4 agosto 2025, granularità 5 minuti)



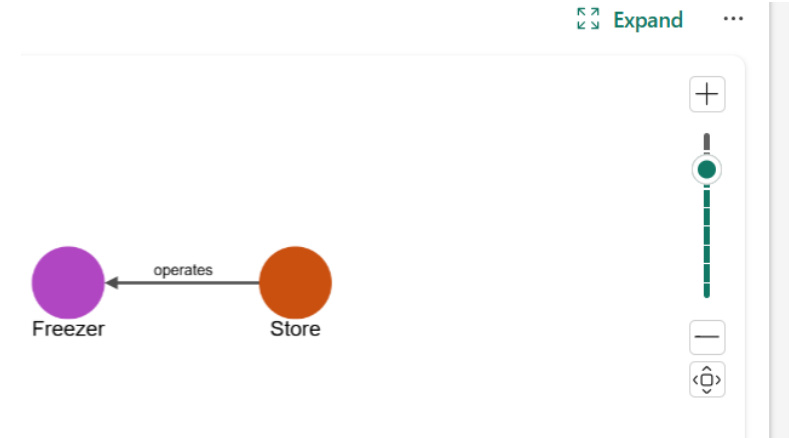
ProductId	RevenueUSD	SaleDate	SaleId	StoreId	Units
P-ICE-002	150	2025-08-04T00:00:00...	8492878	S-BER-01	25
P-ICE-001	155	2025-08-05T00:00:00...	2637459	S-AMS-01	31
P-ICE-002	132	2025-08-02T00:00:00...	1746483	S-PAR-01	22
P-ICE-003	108	2025-08-03T00:00:00...	2637458	S-AMS-01	18
P-ICE-001	170	2025-08-01T00:00:00...	1746482	S-PAR-01	34
P-ICE-001	140	2025-08-01T00:00:00...	8492877	S-BER-01	28



<https://learn.microsoft.com/it-it/fabric/iq/ontology/tutorial-3-preview-ontology>

Il grafico delle relazioni e le query

- Dalla scheda Overview, Espandi il Grafico relazione per vedere nodi e archi tra le entità collegate
- **Query di esempio 1:** tutti i freezer del negozio di Parigi (filtro StoreId = S-PAR-01)
- **Query di esempio 2:** tutti i negozi con una vendita di ricavo superiore a 150 (filtro RevenueUSD > 150)
- Il Generatore di query permette di aggiungere nodi, archi e filtri prima di eseguire la query





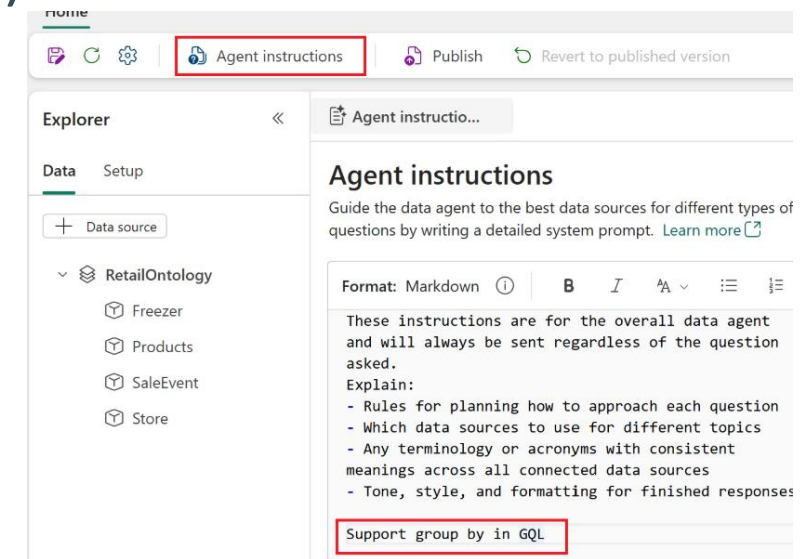
Demo

Parte 4 di 4 — Usare l'ontologia dagli agenti



Creare il data agent

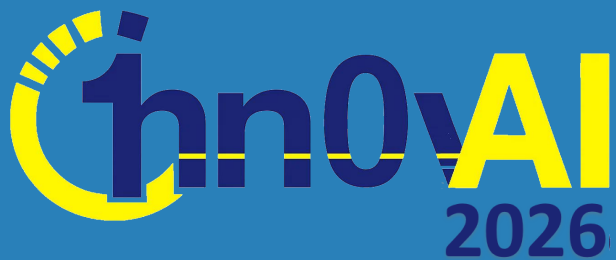
- + Nuovo elemento → Agente dati → nome **RetailOntologyAgent**
- Aggiungi un'origine dati → cercare e selezionare **RetailSalesOntology**
- L'ontologia e i suoi tipi di entità diventano visibili nell'Explorer dell'agente
- **Istruzioni agente:** aggiungere "Support group by in GQL" per migliorare l'aggregazione delle query (risolve un problema noto)



Interrogare l'ontologia in linguaggio naturale

- **Prompt di esempio 1:** *For each store, show any freezers operated by that store that ever had a humidity lower than 46 percent.*
- **Prompt di esempio 2:** *What is the top product by revenue across all stores?*

Se compaiono errori "nessun dato", attendere qualche minuto per l'inizializzazione dell'agente e riprovare



Demo

Cosa succede senza ontologie?



TD SYNnex



DITEDI



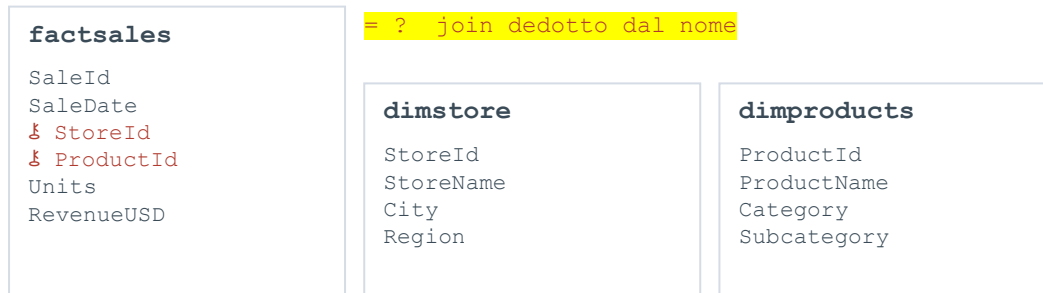
UNIPORDENONE
moltiplica il valore della formazione

La stessa domanda, con e senza ontologia

Le tabelle sono identiche: `dimstore`, `dimproducts`, `factsales` nel lakehouse. Quello che cambia è **dove vive il significato**: nella testa di chi scrive la query (o nel prompt dell'agente), oppure in uno strato governato che chiama le cose `Store`, `Products`, `SaleEvent` e **dichiara che una Sales avviene from un store e ha sold un product.**

SENZA ONTOLOGIA

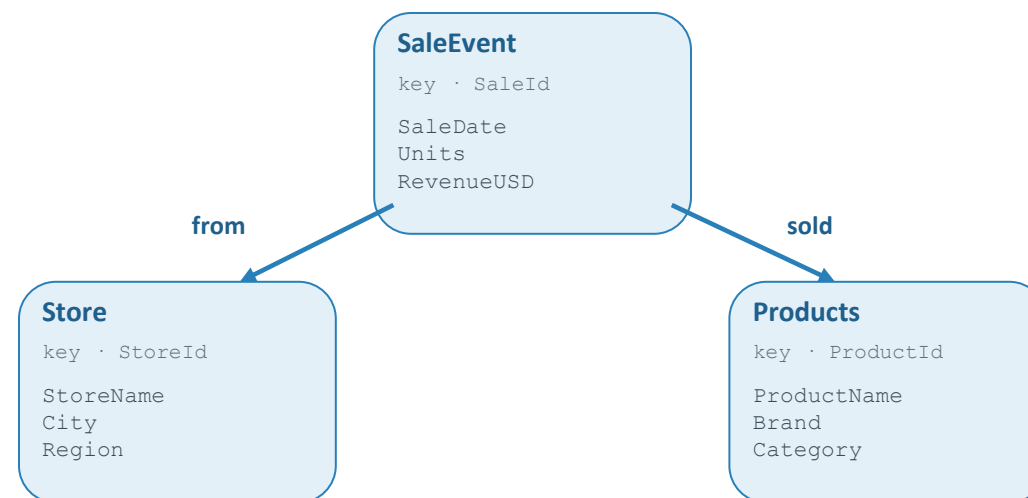
tabelle grezze nel lakehouse



Nessuna chiave dichiarata sulla fact. Nessuna relazione dichiarata. "Vendita", "negozio", "regione" esistono solo nella testa di chi interroga.

CON ONTOLOGIA

RetailSalesOntology



Entità con chiave, relazioni con nome e verso, binding fatti una volta. Chi interroga (persona, report, agente) eredita la stessa definizione.

Domanda 1 · Ricavi per regione

How much did we sell by region from August 1 to August 7, 2025?

SENZA ONTOLOGIA

cosa deve indovinare chi interroga

- "Vendite" = factsales? Il nome lo suggerisce, nulla lo dichiara. "Venduto" è RevenueUSD o Units?
- factsales.StoreId → dimstore.StoreId: il join si deduce perché le colonne si chiamano uguali.
- Chi lo sa? L'analista che ha costruito il modello. La conoscenza sta nella sua testa o nel prompt.

```
SELECT
  d.Region,
  SUM(RevenueUSD) AS total_sales
FROM factsales as f
INNER join dimstore as d on f.StoreId=d.StoreId
WHERE SaleDate >= CAST('2025-08-01' as date)
  AND SaleDate < cast('2025-08-08' as date)  -- makes 1-7 inclusive
GROUP BY Region
ORDER BY Region;
```

Regione	Ricavi USD
France	302 USD
Germany	290 USD
Netherlands	263 USD

CON ONTOLOGIA

cosa è già dichiarato

- "Vendite" è l'entity type SaleEvent, con chiave dichiarata SaleId.
- "Regione" è Store.Region, raggiungibile da SaleEvent tramite la relazione from.
- Il join è un binding fatto una volta in fase di modellazione, non una scelta di chi interroga.

```
MATCH (s:SaleEvent)-[:sold]->(st:Store)
FILTER s.SaleDate >= ZONED_DATETIME('2025-08-01T00:00:00Z')
  AND s.SaleDate < ZONED_DATETIME('2025-08-08T00:00:00Z')
LET Region = st.Region
RETURN Region, SUM(s.RevenueUSD) AS total_sales
GROUP BY Region
ORDER BY Region
```

Regione	Ricavi USD
France	302 USD
Germany	290 USD
Netherlands	263 USD

Stesso risultato. L'ontologia non cambia il numero: elimina le tre assunzioni che servivano per ottenerlo.

Domanda 2 · Negozi che vendono Sorbet

Which stores sold products in the Sorbet category, and how many units did each sell?

SENZA ONTOLOGIA

cosa deve indovinare chi interroga

- Due join da dedurre: factsales → dimstore e factsales → dimproducts. Nessuno dei due è dichiarato.
- "Categoria": Category o Subcategory? Solo chi conosce i dati sa che "Sorbet" è una Category.
- Il verso del join è implicito: se domani ci fosse anche ShipFromStoreId, quale StoreId è "il negozio che ha venduto"?

```
SELECT d.StoreName, SUM(f.Units) AS Units
FROM   factsales f
JOIN   dimstore   d ON d.StoreId   = f.StoreId   -- dedotto
JOIN   dimproducts p ON p.ProductId = f.ProductId -- dedotto
WHERE  p.Category = 'Sorbet'
GROUP BY d.StoreName ORDER BY Units DESC;
```

Regione	Quantità
Lakeshore Retail Amsterdam	18

CON ONTOLOGIA

cosa è già dichiarato

- from e sold sono relationship type con origine e target dichiarati: SaleEvent → Store, SaleEvent → Products.
- "Categoria" è la proprietà Category di Products; il semantic enrichment può descriverne i valori.
- La stessa traversata Store ← SaleEvent → Products serve al report, al data agent e a chi esplora il grafo.

```
MATCH (s:Store)-[]-(e:SaleEvent)-[]-(p:Products) WHERE p.Category =
'Sorbet' RETURN s.StoreName AS StoreName, SUM(e.Units) AS Units GROUP BY
StoreName ORDER BY Units DESC
```

Regione	Quantità
Lakeshore Retail Amsterdam	18

Stesso risultato. Il percorso Store ← SaleEvent → Products è già nel modello: la domanda si limita a percorrerlo.

Cosa cambia, in sintesi

	SENZA ONTOLOGIA	CON ONTOLOGIA
Vocabolario	Nomi tecnici (factsales, dimstore). Ogni team li traduce a modo suo.	Concetti di business (SaleEvent, Store) definiti una volta e riutilizzati ovunque.
Join	Logica riscritta in ogni query; la FK va dedotta dal nome delle colonne.	Relazioni tipizzate e con verso (from, sold) legate ai dati una sola volta.
Chiavi	Implicite: factsales non ha una chiave dichiarata.	Ogni entità ha una entity type key esplicita (SaleId, StoreId, ProductId).
Sorgenti	Ogni sistema ha il suo linguaggio: T-SQL, KQL, DAX.	I binding collegano lakehouse, eventhouse e semantic model allo stesso concetto.
Agenti AI	L'agente indovina significato e join dai nomi, o va istruito prompt per prompt.	L'agente ragiona sui concetti: "vendite per regione" è già SaleEvent → from → Store.Region.
Governance	Definizioni sparse in query, report e documenti; divergono nel tempo.	Un unico posto governato dove nomi, tipi, vincoli e lineage sono dichiarati.

La trappola

factsales.Region è la regione di consegna degli ordini online. La regione del negozio sta in dimstore.Region. Stesso nome, stesso tipo, altro significato.

Tre vendite di Parigi consegnate in Germania e Olanda

Domanda 1 · Ricavi per regione

How much did we sell by region from August 1 to August 7, 2025?

TRAPPOLA · COLONNA OMONIMA

SENZA ONTOLOGIA
T-SQL generato dai nomi delle colonne

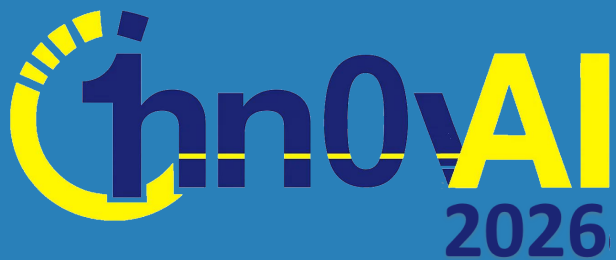
Regione	Ricavi USD
Netherlands	170
Germany	503
France	170

● Classifica invertita: la Francia, prima regione per ricavi, risulta ultima. Nessun errore, nessun avviso.

CON ONTOLOGIA
SaleEvent → from → Store.Region

Regione	Ricavi USD
Netherlands	263
Germany	362
France	542

● Nel modello "Region" esiste una volta sola, su Store. La colonna della fact è bindata come **SDeliveryRegion**.



l'arricchimento semantico?



Semantic enrichment: il pezzo che serve all'AI

Aggiunge metadati strutturati agli oggetti dell'ontologia per migliorare la reperibilità, dare contesto agli agenti e garantire una comprensione uniforme in azienda.

Description

Spiega scopo e significato di entità, proprietà e relazioni.

Entità • Proprietà • Relazioni

Synonyms

Nomi alternativi con cui gli utenti si riferiscono allo stesso concetto.

Solo entità

Additional metadata

Coppie chiave-valore di dominio: unità di misura, sensibilità, owner, qualità del dato.

Entità • Proprietà • Relazioni

Metadati semantici: cosa sono e dove agiscono

Non cambiano la query: cambiano come l'agente legge lo schema prima di scriverla. Aggiunge metadati strutturati agli oggetti dell'ontologia per migliorare la reperibilità, dare contesto agli agenti e garantire una comprensione uniforme in azienda.

	Descrizione	Sinonimi	Metadati k/v
Entity type	✓	✓	✓
Property	✓	–	✓
Relationship type	✓	–	✓

Esempi di metadati k/v: Unit of measurement: cm · Sensitivity: Confidential · Business owner: nome · Data quality: Incomplete. **Le chiavi devono essere uniche per oggetto. Esempio** Esempi di metadati utili:
unit: celsius
Sensitivity: Confidential

DOVE ENTRANO NEL LAVORO DELL'AGENTE

- 1 Legge lo schema dell'ontologia**
entità, proprietà, relazioni + descrizioni, sinonimi, metadati
- 2 Collega le parole della domanda ai concetti**
"frozen desserts" → Products, "country" → Store.Region
- 3 Genera la query sull'ontologia**
i metadati qui non intervengono
- 4 Esegue e risponde**
unità, valute e definizioni arrivano dal passo 1

Limite attuale: l'arricchimento aiuta l'agente nell'esplorazione dello schema e nel ragionamento, ma non entra direttamente nella generazione della query sull'ontologia.

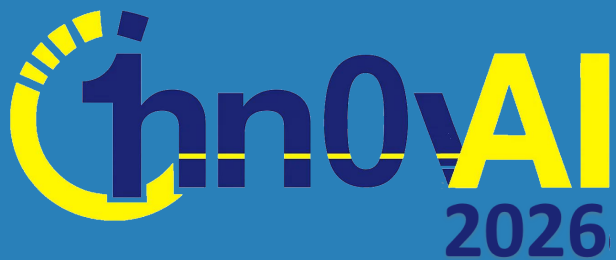
L'effetto misurabile sull'agente

Domanda: *"Which ice cream shops performed best this month and sold the most frozen desserts?"*

Prima dell'arricchimento il data agent non riesce a rispondere: nell'ontologia non esiste nulla chiamato "dolci surgelati (frozen desert)".

Best practice

- Descrizioni brevi (una-tre frasi) che partono da cosa rappresenta l'oggetto
- Sinonimi con abbreviazioni, acronimi, gergo di settore e varianti regionali
- Unità di misura sulle proprietà numeriche e classificazioni di sensibilità
- Convenzioni di naming delle chiavi coerenti e documentate per il team
- Manutenzione nel tempo: la terminologia aziendale cambia



Demo

arricchimento semantico



Prima e dopo: l'esempio ufficiale Lakeshore

Domanda al data agent: *"Which ice cream shops performed best this month and sold the most frozen desserts?"*

SENZA ENRICHMENT

- **"The data agent can't retrieve the requested information"**
Nell'ontologia non esiste nulla che si chiami frozen desserts né ice cream shop: le parole della domanda non trovano un concetto.

Cosa manca

- Products si chiama Products: nessun legame con "desserts", "treats", "menu items"
- Category è una stringa senza spiegazione: "frozen" non porta da nessuna parte
- "shops" non è un sinonimo di Store

L'agente non è impreciso: non ha un vocabolario. Sa che esiste Products, non sa che un gelato è un prodotto.

CON ENRICHMENT

Products · sinonimi

frozen desserts · ice cream items · desserts · menu items · treats · products sold

Products · metadati

seasonality: summer · shelfPlacement: frozen · requiresFreezerTruck: true

Products.Category · descrizione

"Represents the high-level group or classification"

Products.Category · metadati

supportSalesReporting: true · promotionPriority: high

- "The data agent better relates the terms in the question to data in the ontology and can answer the question."

Senza metadati: tre risposte sbagliate con sicurezza

Sui dati della demo. Nessun errore a schermo: l'agente risponde, e risponde male.

minSafeTempC

Which freezers were above their safety threshold on August 8?

- Il nome dice "min": l'agente legge la soglia come un minimo e segnala i freezer sotto -18 °C, cioè quelli sani. Risposta invertita.

- **Descrizione:** "Maximum temperature allowed for safe storage; a reading above this value is a violation"

Metadato: unit: °C

Region vs DeliveryRegion

How much did we sell per country?

- Nessuna proprietà si chiama country. Due candidate: Store.Region e SaleEvent.DeliveryRegion. L'agente sceglie la più vicina alla fact.

- **Descrizione Store.Region:** "Country where the store is located"
Descrizione DeliveryRegion: "Country the online order was shipped to, not the store's"

RevenueUSD

What was the revenue in euros last week?

- Il numero in dollari viene presentato come euro, oppure l'agente inventa un tasso di cambio.

- **Metadato:** currency: USD
Descrizione: "Gross revenue in US dollars; no conversion available"

Altri quattro casi, e tre limiti da conoscere

PROMPT	SENZA METADATI	METADATO CHE LO RISOLVE
<i>How many pints did we sell in Berlin?</i>	"Pint" è un valore di Subcategory: l'agente cerca una proprietà pints e non la trova, o somma Units senza filtro.	Descrizione su Subcategory ("Package format: Pint") e su Units ("Number of pint containers sold")
<i>Which stores had their freezer door open for too long?</i>	doorOpen è 0/1: viene trattato come conteggio o come durata.	Descrizione: "1 = door open at reading time; a flag, not a duration"
<i>Show me the best-selling category</i>	"category" è anche parola generica: l'agente raggruppa per Subcategory o Brand.	Descrizione su Category ("High-level product group: Ice Cream, Sorbet")
<i>Which ice cream shops sold the most treats?</i>	Nessun sinonimo: "can't retrieve the requested information".	Sinonimi su Store (shop, point of sale) e Products (treats, desserts)

TRE LIMITI DICHIARATI NELLA DOCUMENTAZIONE

- Agisce solo sull'interpretazione dello schema — se la query è sbagliata per altri motivi (dati duplicati, join), i metadati non la correggono.
- **I sinonimi esistono solo sugli entity type** — per proprietà e relazioni si usa la descrizione.
- **L'enrichment sulle relazioni non è ancora usato dai data agent** — descrivere from come "store where the sale happened" serve alla governance, non all'agente.

Conclusioni

- Il significato esce dai report ed entra nel modello – entità, relazioni e regole di business definite una volta sola, riusabili da tutti.
- L'AI risponde bene solo se capisce il business – con dati "sporchi" o il data agent sbaglia con sicurezza; l'ontologia lo guida verso l'interpretazione corretta.
- Un'ontologia, molti consumatori – data agent, operations agent, Copilot Studio, Foundry IQ e client MCP leggono la stessa semantica.
- Non sostituisce il data modeling, lo presuppone – qualità dei dati e governance restano il prerequisito.

le ontologie descrivono il business, i data agent lo rendono interrogabile: senza la prima, la seconda è solo una SQL più veloce a sbagliare.